УДК 519.872

Р.С. Хабаров
[степень, звание, место работы — заполнить]

ТОЧНАЯ ВЕРОЯТНОСТЬ НАРУШЕНИЯ SLA В СИСТЕМАХ ПАКЕТНОГО ОБСЛУЖИВАНИЯ С ОГРАНИЧЕННЫМ РАЗМЕРОМ ПАКЕТА И ФАЗОВЫМ ВРЕМЕНЕМ ОБСЛУЖИВАНИЯ

Рассмотрена система пакетного (batch-service) обслуживания со временем обслуживания пакета, зависящим от его размера, — модель динамического батчинга при инференсе нейронных сетей, в том числе больших языковых моделей (LLM). Предложен метод получения точной (не приближённой и не являющейся оценкой сверху) вероятности нарушения соглашения об уровне обслуживания (SLA) $P(W > D)$ для непрерывного времени, ограниченного максимального размера пакета и фазового (E_k/H2, произвольный коэффициент вариации) времени обслуживания, зависящего от размера пакета. Метод основан на PASTA-разложении времени ожидания по фазовому типу и действии матричной экспоненты на явно построенный подгенератор. Результат точно регрессирует к известным частным случаям и подтверждён независимым имитационным моделированием. Численное сравнение с опубликованной оценкой сверху среднего времени ожидания показывает, что при реалистичном максимальном размере пакета истинное среднее время ожидания может превышать эту оценку почти на порядок; сравнение с приближением по моментам показывает систематическое занижение вероятности нарушения SLA в глубоком хвосте на несколько порядков.

Ключевые слова: теория массового обслуживания, пакетное обслуживание, обслуживание, зависящее от размера пакета, фазовые распределения, SLA, вероятность нарушения дедлайна, инференс больших языковых моделей, динамический батчинг.

EXACT SLA-VIOLATION PROBABILITY FOR BATCH-SERVICE QUEUES WITH A BOUNDED BATCH SIZE AND PHASE-TYPE SERVICE TIME

R.S. Khabarov

A batch-service queue with batch-size-dependent service time is the natural model for dynamic-batching neural-network (including large language model, LLM) inference. We derive an exact — not approximate, not an upper bound — deadline-violation probability $P(W > D)$ for continuous time, a bounded maximum batch size and phase-type (E_k/H2, arbitrary coefficient of variation) batch-size-dependent service time, via matrix-exponential action on an explicitly constructed phase-type sub-generator obtained from a PASTA decomposition of the waiting time. The result regresses exactly to known degenerate cases and is validated against independent discrete-event simulation. A numerical comparison with a published closed-form upper bound on mean wait shows that under a realistic bounded batch size the true mean wait

can exceed that bound by almost an order of magnitude; comparison with a moment-fit approximation shows systematic underestimation of deep-tail SLA-violation probability by several orders of magnitude.

Key words: queueing theory, batch service, batch-size-dependent service, phase-type distributions, SLA, deadline-violation probability, large language model inference, dynamic batching.

Введение

Ускорение инференса нейронных сетей на графических процессорах (GPU) существенно опирается на динамический батчинг — объединение нескольких независимо поступивших запросов в один пакет (batch), обрабатываемый сервером одновременно. Такое объединение повышает пропускную способность и энергоэффективность сервера, но ценой дополнительной задержки: запрос вынужден дожидаться, пока сформируется пакет и пока этот пакет будет обслужен [1]. Применительно к инференсу больших языковых моделей (LLM) данная задержка прямо входит в метрику времени до первого токена (time-to-first-token, TTFT), для которой операторы всё чаще формулируют требования к качеству обслуживания (SLO) в терминах вероятности, а не среднего значения: $P(\text{TTFT} > D) < \varepsilon$ для заданного малого ε [2; 3].

Два структурных свойства реальных систем динамического батчинга отличают их от классической модели пакетного обслуживания Бейли [4]. Во-первых, время обслуживания пакета зависит от его размера: обработка пакета из b заявок занимает больше времени, чем пакета из одной заявки, но не пропорционально b — типичная эмпирическая зависимость близка к линейной с положительным свободным членом [1]. Во-вторых, размер пакета ограничен сверху: память GPU ограничивает максимальный размер батча $b_{\max}$, и на практике $b_{\max}$ — конечное, зачастую небольшое число, а не абстрактная бесконечность.

Прямой практический прецедент — работа Inoue [1], формулирующая сервер инференса как batch-service очередь с зависящим от размера пакета временем обслуживания и выводящая аналитическую оценку сверху среднего времени ожидания $E[W]$ в замкнутой форме при двух упрощающих допущениях: неограниченный максимальный размер пакета (при освобождении сервера в пакет забираются все ожидающие заявки) и детерминированное, линейно зависящее от размера время обслуживания пакета. Автор прямо указывает, что точный анализ труден даже для зависящего от размера пакета $M/M/1$, и что для случая конечного $b_{\max}$ известен лишь алгоритмический, не дающий замкнутой формулы подход на основе матрично-аналитических методов [5].

Начиная с работы [6] и продолжая обширной серией последующих работ [7—10], детально проработано точное стационарное распределение длины очереди (и содержимого сервера) для систем пакетного обслуживания с зависящим от размера пакета, в том числе фазовым, временем обслуживания при различных входных потоках. Эти работы дают точное распределение длины очереди в различные моменты времени, но не распределение времени ожидания конкретной

помеченной заявки. Классический результат для времени ожидания в системе пакетного обслуживания с общим, но не зависящим от размера пакета временем обслуживания получен точно методом корней характеристического уравнения [11]; та же группа авторов, расширяя метод на коррелированный входной поток, в 2025 г. явно называет полученное распределение времени ожидания эвристическим приближением [12] — то есть даже без зависимости от размера пакета точное решение для коррелированного входа остаётся открытым. Единственная найденная нами работа, дающая хвост времени ожидания при обслуживании, зависящем от размера пакета, — [13], но это дискретное время и заявленное самими авторами приближение.

Таким образом, пересечение условий «точно» × «непрерывное время» × «время ожидания, а не только длина очереди» × «обслуживание, зависящее от размера пакета» не закрыто ни одной из найденных работ, включая самые свежие публикации 2024–2026 гг. той же исследовательской традиции. Дальнейшая структура статьи следующая. Сначала приводится постановка задачи. Далее описан метод: PASTA-разложение времени ожидания по фазовому типу и точный хвост через действие матричной экспоненты. После этого представлены результаты проверки и два численных эксперимента — сравнение с оценкой [1] и с приближением по моментам.

Постановка задачи

Рассматривается однолинейная система $M/PH^{[1,b]}/1$: пуассоновский входной поток интенсивности λ; как только сервер освобождается и есть хотя бы одна ожидающая заявка, формируется пакет из $\min(b, j)$ заявок, где j — число ожидающих (минимальный порог запуска $a = 1$, то есть сервер не ждёт накопления нескольких заявок — типичный режим динамического батчинга); время обслуживания пакета размера i имеет фазовое распределение — E_k (Эрланга порядка k в нотации Кендалла) с темпом фазы $\text{rate}(i)$ либо $\text{H2}(p_1(i),\ \mu_1(i),\ \mu_2(i))$ — с произвольной зависимостью параметров от i, в частности эмпирически наблюдаемой линейной зависимостью среднего от размера пакета [1].

Цель — получить точное $P(W > D)$ для $D \geq 0$, где W — время ожидания в очереди случайно выбранной (по PASTA, Poisson Arrivals See Time Averages) заявки, без ограничения на распределение времени обслуживания, кроме принадлежности фазовому классу.

Метод

Система моделируется усечённой по длине очереди ($j \leq N$) цепью Маркова с состояниями (i, φ, j), где $i \in \{0, \ldots, b\}$ — размер обслуживаемого пакета ($i = 0$ — простой), φ — текущая фаза обслуживания пакета (для E_k — номер пройденной фазы $0, \ldots, k-1$, для H2 — выбранная при старте ветвь $\{0, 1\}$), j — число ожидающих заявок. Стационарное распределение $\pi(i, \varphi, j)$ находится прямым решением $\pi Q = 0$ для разреженного генератора Q.

По свойству PASTA заявка, приходящая в систему, видит стационарное рас-

пределение $\pi(i, \varphi, j)$. Если сервер простаивает ($i = 0$), заявка немедленно формирует пакет: $W = 0$. Если сервер занят, время ожидания раскладывается на два независимых слагаемых: остаток текущего обслуживаемого пакета (E_{k-p} с темпом rate(i) для случая E_k, где p — застанная фаза; Exp$(\mu_\varphi(i))$ для H2-случая по марковости экспоненциальной фазы) и $\lfloor j/b \rfloor$ полных последующих пакетов размера b. Для E_k это строго последовательная цепь из $\lfloor j/b \rfloor \cdot k$ фаз ставки rate(b); для H2 — ветвящаяся цепь, поскольку каждый последующий пакет независимо заново выбирает свою ветвь.

Для каждого занятого состояния (i, φ, j) с $\pi(i, \varphi, j) > 0$ строится явный конечный подгенератор $T_{i,\varphi,j}$ поглощающей цепи Маркова, представляющей это разложение (начальный вектор $\alpha = e_1$, поглощение — момент начала обслуживания помеченной заявки). Условный хвост времени ожидания — классическая формула phase-type распределения:

$$P(W > t \mid i, \varphi, j) = \alpha \exp(T_{i,\varphi,j}\, t)\, \mathbf{1}. \tag{1}$$

Итоговый, безусловный точный хвост:

$$P(W > t) = \sum_{\substack{(i,\varphi,j):\\ i \geq 1}} \pi(i, \varphi, j)\, \alpha \exp(T_{i,\varphi,j}\, t)\, \mathbf{1}. \tag{2}$$

Для $M/M/1$ и $M/E_k^{[1,b]}/1$ подгенератор T — двухдиагональная (последовательная) матрица; для H2 он содержит ветвление. Экспонента вычисляется не явным построением плотной матрицы, а её действием на единичный вектор через разреженное представление, что даёт точный результат без ручного разбора случаев совпадающих/несовпадающих кратностей ставок, неизбежного при классическом подходе частичных дробей для гипоэкспоненциальных/Coxian распределений.

При $b = 1$ (отсутствие батчинга) результат должен точно совпасть с классической формулой хвоста $M/M/1$: $P(W > t) = \rho \exp(-\mu(1 - \rho)t)$. При $k = 1$ E_k-калькулятор должен точно совпасть с экспоненциальным (MM1) калькулятором; при $p_1 = 1$ H2-калькулятор должен точно совпасть с ним же (вторая ветвь становится недостижимой). Эти тождества — не только регрессионные тесты, но и необходимое условие корректности построения.

Проверка

Все три регрессионных тождества подтверждены с точностью до ошибок округления float64 (порядка 10^{-16}). Независимо от регрессий результат для обслуживания, зависящего от размера пакета, где регрессионные тождества неприменимы, проверен против независимого имитационного моделирования — дискретно-событийной симуляции, реализованной независимо от кода основного метода, — для каждого из трёх калькуляторов, в нескольких точках t, включая точки глубокого хвоста, где симуляция из 3×10^6 выборок не дала ни одного события превышения порога: точный результат в этих точках даёт малое, но

ненулевое значение, согласующееся по порядку величины с границей, которую могла бы дать симуляция существенно большего объёма.

Численные эксперименты

Первая реализация использовала явное построение плотной матричной экспоненты на подгенераторе размера до нескольких сотен состояний (при большом $\lfloor N/b \rfloor$). Время одного вызова составило 57 секунд при реалистичном усечении очереди ($N = 500$). Переход на действие экспоненты на вектор без явного построения плотной матрицы дал идентичный результат за 0,19 секунды — ускорение около 300 раз, необходимое условие практического применения метода.

Для сравнения с оценкой [1] использованы параметры эмпирической линейной подгонки автора для батч-инференса ResNet-50 на Tesla V100 ($\alpha = 0{,}1438$, $\tau_0 = 1{,}8874$), $\lambda = 0{,}8$ (устойчивая система при любом $b_{\max} \geq 2$: максимальная пропускная способность при $b_{\max} = 2$ составляет $2/(2\alpha + \tau_0) \approx 0{,}92$). Аналитическая оценка [1] в замкнутой форме для неограниченного батча: $\varphi(\lambda, \alpha, \tau_0) = 3{,}353$.

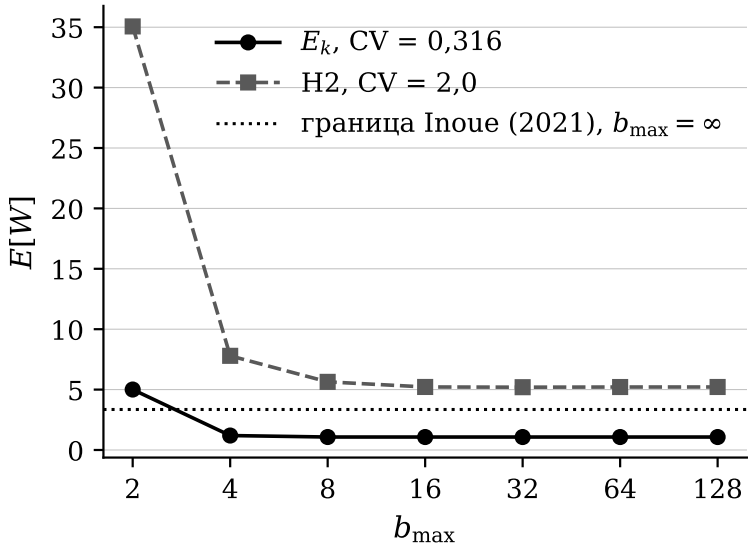


Рисунок 1. Истинное среднее время ожидания $E[W]$ в зависимости от $b_{\max}$ против границы [1] ($\lambda = 0{,}8$)

При малом (реалистичном для памяти GPU) $b_{\max}$ истинное среднее время ожидания превышает закрытую оценку [1] — несмотря на то, что последняя формально является оценкой сверху: граница корректно доказана для неограниченного батча и не распространяется на ограниченный случай. При коэффициенте вариации $CV = 2$ (разумное предположение для пакета с запросами, смешанными по длине, типичными для LLM-инференса) расхождение при $b_{\max} = 2$ достигает почти десятикратного значения. При больших $b_{\max}$, приближающихся к режиму [1], точное значение остаётся ниже границы — граница верна именно там, где и была выведена, но не несёт информации о своей неприменимости при малом $b_{\max}$.

Поскольку параметры [1] получены на ResNet-50-инференсе 2021 года, для независимой проверки того же качественного вывода на современном LLM-

serving использованы измеренные данные Bench360 [14] (offline-batch throughput SGLang на GPU L4, таблица 4 источника): из зависимости токенов/с от размера батча $B \in \{16, 32, 64, 128\}$ при фиксированной длине генерации восстановлена линейная аппроксимация $g(s) = \alpha s + \tau_0$ методом наименьших квадратов, $\alpha \approx 0{,}076$ с, $\tau_0 \approx 6{,}30$ с — на порядок больший фиксированный "оверхед" на партию, чем в [1], что ожидаемо для декодирующего цикла LLM по сравнению со свёрточной сетью. При $\lambda = 0{,}25$ (80% от пропускной способности при $b_{\max} = 2$) точное $E[W]$ по-прежнему превышает соответствующую границу [1] при малом $b_{\max}$ (10,24 против 9,72 при $b_{\max} = 2$) и быстро сходится к $\approx 3{,}11$ уже при $b_{\max} = 8$ — то же качественное поведение, что и на синтетических параметрах, на независимо измеренных данных другого поколения оборудования и другого класса рабочей нагрузки.

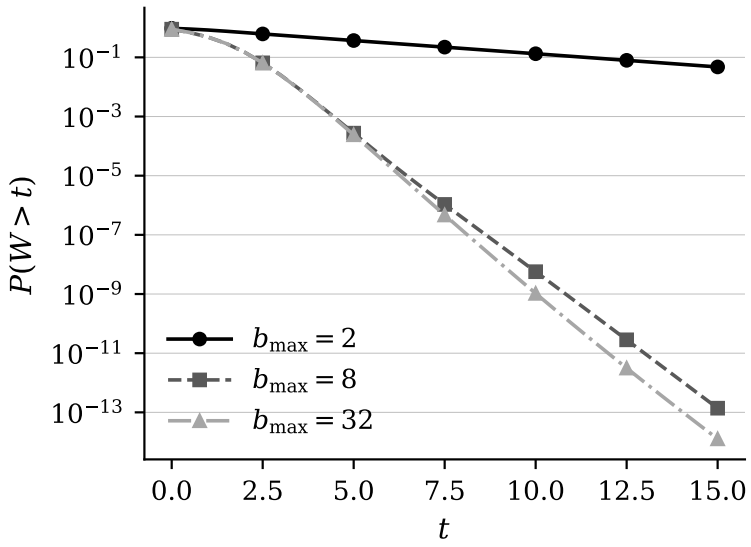


Рисунок 2. Точный хвост $P(W > t)$ для нескольких $b_{\max}$ (E_k, $CV = 0{,}316$)

На рисунке 2 видно, что форма хвоста существенно зависит от $b_{\max}$: при $b_{\max} = 2$ (режим, близкий к границе устойчивости при выбранной нагрузке) хвост спадает заметно медленнее, чем при $b_{\max} = 8$ и $b_{\max} = 32$, кривые для которых практически совпадают — система уже близка к своему пределу при неограниченном батче.

Для случая E_k ($CV = 0{,}316$) сравнивается точный хвост с результатом стандартного приёма «подгонка 2–3 моментов под H2/Gamma, вычисление хвоста подогнанного распределения», широко используемого на практике, когда точный метод недоступен.

Таблица 1. Точная и подогнанная по моментам вероятность $P(W > D)$

$b_{\max}$	D	Точная $P(W > D)$	Подогнанная (2 момента)	Отн. ошибка
2	5	$3{,}7035 \times 10^{-1}$	$3{,}7107 \times 10^{-1}$	$+0{,}19\%$
2	10	$1{,}3255 \times 10^{-1}$	$1{,}3331 \times 10^{-1}$	$+0{,}57\%$
2	20	$1{,}6981 \times 10^{-2}$	$1{,}6906 \times 10^{-2}$	$-0{,}44\%$
8	5	$2{,}7386 \times 10^{-4}$	$2{,}5296 \times 10^{-3}$	$+824\%$
8	10	$5{,}6797 \times 10^{-9}$	$2{,}4628 \times 10^{-6}$	$+43\,261\%$
8	20	$3{,}3642 \times 10^{-18}$	$1{,}6570 \times 10^{-12}$	$+4{,}93 \times 10^{7}\%$
16	5	$2{,}4137 \times 10^{-4}$	$2{,}5019 \times 10^{-3}$	$+937\%$
16	10	$1{,}0582 \times 10^{-9}$	$2{,}4014 \times 10^{-6}$	$+226\,833\%$
16	20	$5{,}7739 \times 10^{-19}$	$1{,}5685 \times 10^{-12}$	$+2{,}72 \times 10^{8}\%$

При умеренно глубоком хвосте ($b_{\max} = 2$, где D лишь в несколько раз превышает среднее) подгонка по двум моментам работает приемлемо (ошибка менее 1%). Но при малом среднем времени ожидания и дедлайне, уходящем в действительно глубокий хвост — во много раз больше среднего, типичный для GPU/LLM SLA-режима, где цель — $P(\text{нарушение}) < 0{,}1\%$ — подгонка по моментам систематически и резко переоценивает вероятность нарушения SLA: на один-два порядка при умеренной глубине хвоста и на много порядков при истинно глубоком хвосте.

Практические следствия: выбор $b_{\max}$ и предел приближений

Тождество баланса потоков. Для любой стабильной эргодической системы пакетного обслуживания с $E[\text{длительность пакета} \mid \text{размер } s] = \alpha s + \tau_0$ (верно по построению и для E_k-, и для H2-калькулятора, независимо от распределения длительности при фиксированном размере) коэффициент загрузки сервера удовлетворяет точному тождеству

$$\rho = \lambda\alpha + \frac{\lambda\tau_0}{\bar{S}}, \tag{3}$$

где $\bar{S}$ — средний размер пакета, усреднённый по пакетам (не по заявкам). Тождество следует из renewal-reward: интенсивность завершения пакетов в стационарном режиме $r = \lambda/\bar{S}$ (баланс потоков), а $\rho = r \cdot E[g(S)] = r(\alpha\bar{S} + \tau_0)$ по линейности g, без предположений о независимости размера пакета и его длительности.

Поскольку $\lambda\tau_0/\bar{S}$ убывает по $\bar{S}$, монотонность $E[W](b_{\max})$ (видимая на рисунке 1) сводится к монотонности $\bar{S}(b_{\max})$. Прямое pathwise-доказательство этой монотонности наталкивается на реальное препятствие: система с большим $b_{\max}$ иногда формирует один большой и поэтому медленный пакет и временно отстаёт по числу завершённых заявок от системы с меньшим $b_{\max}$ — полное доказательство остаётся открытым (см. roadmap в репозитории most-queue). Тождество (3) тем не менее даёт проверяемый инструмент: обратное восстановление $\bar{S} = \lambda\tau_0/(\rho - \lambda\alpha)$ из уже посчитанной ρ служит независимой сверкой корректности построения CTMC и подтверждено точно на всех $b_{\max}$ обоих вариантов изменчивости (таблица 2).

Таблица 2. Коэффициент загрузки ρ и восстановленный средний размер пакета $\bar{S}$

$b_{\max}$	ρ (E_k)	$\bar{S}$ (E_k)	ρ (H2)	$\bar{S}$ (H2)
2	0,9580	1,79	0,9388	1,83
4	0,8991	1,93	0,8172	2,15
8	0,8948	1,94	0,7802	2,27
16	0,8947	1,94	0,7688	2,31
32	0,8947	1,94	0,7665	2,32

Минимальный $b_{\max}$ под целевой SLA. Точный метод, применённый как чёрный ящик, даёт прямой прескриптивный результат: минимальный $b_{\max}$ — наименьшее требование к памяти GPU на партию — при котором $P(W > D) \leq \varepsilon$ для заданного дедлайна D и целевого уровня ε (таблица 3, случай E_k). Для H2 (высокая изменчивость) ни один из целевых уровней $\varepsilon \in \{10^{-2}, 10^{-3}, 10^{-4}\}$ не достижим ни при каком $b_{\max} \leq 128$ для $D \leq 30$ — высокая изменчивость службы обслуживания фундаментально ограничивает то, чего можно добиться одним лишь укрупнением батчей; нужны либо более мягкий дедлайн, либо снижение изменчивости времени обслуживания другими средствами.

Таблица 3. Минимальный $b_{\max}$, при котором $P(W > D) \leq \varepsilon$ (случай E_k)

D	$\varepsilon = 10^{-2}$	$\varepsilon = 10^{-3}$	$\varepsilon = 10^{-4}$
5	4	8	—
10	4	4	4
20	4	4	4
30	2	4	4

При существенно более глубоком дедлайне ($D = 50$) данные вскрывают контринтуитивный, но воспроизводимый (устойчивый к удвоению усечения очереди N) эффект: точный хвост немонотонен по $b_{\max}$ — $P(W > 50) \approx 7 \times 10^{-46}$ при $b_{\max} = 8$–16, но вырастает до $\approx 1,3 \times 10^{-39}$ при $b_{\max} = 32$ (семь порядков хуже), хотя при умеренных дедлайнах больший $b_{\max}$ всегда не хуже. Причина — та же, что ограничивает ценность асимптотики по $t \to \infty$: истинная скорость экспоненциального затухания хвоста равна $\mathrm{rate}(b_{\max})$ и убывает с ростом $b_{\max}$; при достаточно большом D это в конце концов перевешивает выигрыш от более редких партий. На всех испытанных целевых уровнях ε этот эффект не меняет практический ответ (провал остаётся на много порядков ниже любого разумного ε), но является строгим контрпримером к гипотезе о всеобщей монотонности.

Предел однополюсных приближений. Claeys et al. [13] и предшествующая линия их работ оценивают хвост через доминирующую особенность (полюс, ближайший к началу координат) производящей функции задержки — технику, по духу эквивалентную одной экспоненте, откалиброванной по истинной асимптотической скорости затухания. Точная формула их batch-size-зависимого расширения 2013 года нам недоступна (платный доступ), поэтому вместо попытки

её повторить мы честно реализовали ту же общую технику для нашей модели: $P_{\text{approx}}(W > t) = \rho \exp(-\eta(b_{\max})\, t)$, где η — точная скорость затухания из раздела «Метод», а ρ взято из таблицы 2. Результат (таблица 4) — систематическое и катастрофическое расхождение в обе стороны: при малом $b_{\max}$ однополюсное приближение на много порядков *занижает* хвост (истинная асимптотика ещё не наступила, доминирует полиномиальная "выпуклость" тела распределения); при большом $b_{\max}$ оно на много порядков *завышает* хвост (номинальная ставка $\text{rate}(b_{\max})$ почти никогда не реализуется, поскольку большие партии при лёгкой загрузке формируются редко).

Таблица 4. Точный хвост против однополюсного приближения

Режим	$b_{\max}$	D	Точная	Приближение
E_k	2	10	$1{,}33 \times 10^{-1}$	$1{,}03 \times 10^{-20}$
E_k	128	20	$4{,}74 \times 10^{-19}$	$4{,}69 \times 10^{-5}$
H2	2	20	$5{,}38 \times 10^{-1}$	$4{,}66 \times 10^{-2}$
H2	128	20	$3{,}90 \times 10^{-2}$	$5{,}55 \times 10^{-1}$

Ограничения

Результат ограничен случаем $a = 1$ (нет нижнего порога запуска пакета); случай $a > 1$ требует отдельного («idle-refill») анализа и остаётся открытым. Основное численное сравнение использует параметры подгонки [1] для ResNet-50-инференса; независимая проверка на измеренном throughput современного LLM-serving [14] переносит только среднюю зависимость $g(s) = \alpha s + \tau_0$, а не изменчивость времени обслуживания батча — коэффициент вариации в обоих случаях остаётся синтетическим допущением, — в целом это управляемое численное исследование, а не воспроизведение конкретной производственной системы. Сравнение с оценкой [1] выполнено только по среднему; сравнение с подгонкой по моментам для хвоста выполнено только для случая E_k низкой изменчивости. Монотонность $E[W](b_{\max})$ и $\bar{S}(b_{\max})$ подтверждена численно на всём испытанном диапазоне параметров, но не доказана в общем виде; как показано выше, при достаточно глубоком дедлайне сам точный хвост $P(W > D)$ немонотонен по $b_{\max}$ — общее доказательство (или опровержение) монотонности остаётся открытой задачей.

Заключение

Получена и верифицирована точная вероятность нарушения SLA для системы пакетного обслуживания с ограниченным максимальным размером пакета и фазовым временем обслуживания произвольного коэффициента вариации, зависящим от размера пакета, — результат, закрывающий пересечение условий, не покрытое ни классической линией работ по точному распределению длины очереди для обслуживания, зависящего от размера пакета, ни единственным известным приближённым, дискретно-временным результатом для хвоста

времени ожидания с такой зависимостью. Численно показано, что пренебрежение ограничением размера пакета может занижать истинное среднее время ожидания почти на порядок, а приближение по моментам — занижать точность оценки вероятности нарушения SLA в глубоком хвосте на несколько порядков; оба эффекта напрямую релевантны практике SLA-ориентированного управления GPU/LLM-инференс-серверами. Дополнительно показано, что точный метод даёт прямой прескриптивный инструмент выбора минимального $b_{\max}$ под целевую SLA, и что альтернативная, однополюсная техника оценки хвоста (в духе Claeys et al. [13]) расходится с точным значением на величину до 10^{14}–10^{19} в зависимости от режима.

Список литературы

1. *Inoue Y.* Queueing analysis of GPU-based inference servers with dynamic batching: a closed-form characterization // Performance Evaluation. — 2021. — Т. 147.
2. QUARTZ: Quantile-Aware Routing and Queueing for TTFT SLOs in LLM Serving / Z. Liu [и др.] // Findings of the Association for Computational Linguistics: ACL 2026. — 2026. — С. 37886—37896. — DOI: 10.18653/v1/2026.findings-acl.1888.
3. JITServe: SLO-aware LLM Serving with Imprecise Request Information / W. Zhang [и др.] // Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 2026). — 2026. — arXiv: 2504.20068.
4. *Bailey N. T. J.* On queueing processes with bulk service // Journal of the Royal Statistical Society. Series B. — 1954. — Т. 16. — С. 80—87.
5. *Neuts M. F.* Structured Stochastic Matrices of M/G/1 Type and Their Applications. — New York : Marcel Dekker, 1989.
6. *Banerjee A.*, *Gupta U. C.* Reducing congestion in bulk-service finite-buffer queueing system using batch-size-dependent service // Performance Evaluation. — 2012. — Т. 69, № 1. — С. 53—70.
7. *Pradhan S.*, *Gupta U. C.*, *Samanta S. K.* Queue-length distribution of a batch service queue with random capacity and batch size dependent service: $M/G^{Y}_{r}/1$ // OPSEARCH. — 2016. — Т. 53. — С. 329—343. — DOI: 10.1007/s12597-015-0231-8.
8. *Banerjee A.*, *Gupta U. C.*, *Chakravarthy S. R.* Analysis of a finite-buffer bulk-service queue under Markovian arrival process with batch-size-dependent service // Computers & Operations Research. — 2015. — Т. 60. — С. 138—149. — DOI: 10.1016/j.cor.2015.02.012.
9. *Gupta G. K.*, *Banerjee A.* On $M/G^{(a,b)}/1/N$ queue with batch size- and queue length-dependent service // Mathematics and Computing (ICMC 2018). Т. 253. — Singapore : Springer, 2018. — С. 249—262. — (Springer Proceedings in Mathematics & Statistics). — DOI: 10.1007/978-981-13-2095-8_20.

10. *Pradhan S.*, *Nandy N.*, *Gupta U. C.* Performance analysis of a versatile bulk-service queue with group-arrival, batch-size-dependent service time and queue-length-dependent vacation // Quality Technology & Quantitative Management. — 2024. — Т. 22, № 4. — С. 683—726. — DOI: 10.1080/16843703.2024.2391673.
11. A Novel Computational Procedure for the Waiting-Time Distribution (In the Queue) for Bulk-Service Finite-Buffer Queues with Poisson Input / M. L. Chaudhry [и др.] // Mathematics. — 2023. — Т. 11, № 5. — DOI: 10.3390/math11051142.
12. On the Heuristic Computational Procedures of the Virtual Waiting-Time Distribution in a Non-renewal Input Finite-Buffer Bulk-Service Queues: MAP/$R^{(a,b)}$/1/N / A. Datta Banik [и др.] // Journal of the Indian Society for Probability and Statistics. — 2025. — Т. 26. — С. 585—630. — DOI: 10.1007/s41096-025-00232-0.
13. Tail probabilities of the delay in a batch-service queueing model with batch-size dependent service times and a timer mechanism / D. Claeys [и др.] // Computers & Operations Research. — 2013. — Т. 40, № 5. — С. 1497—1505.
14. *Stuhlmann L.*, *Fadel Argerich M.*, *Fürst J.* Bench360: Benchmarking Local LLM Inference from 360 Degrees // arXiv preprint. — 2026. — arXiv: 2511.16682.

Приложение. Программная реализация

Все вычисления воспроизводимы в открытой библиотеке most-queue: `most_queue.theory.batch.bulk_service.BulkServiceMM1Calc.get_tail`, `most_queue.theory.batch.bulk_service_erlang.BulkServiceErlangCalc.get_tail`, `most_queue.theory.batch.bulk_service_h2.BulkServiceH2Calc.get_tail`. Тесты и независимая проверка имитационным моделированием — `tests/units/test_bulk_service_tail.py`, `test_bulk_service_erlang_tail.py`, `test_bulk_service_h2_tail.py`. Численные эксперименты раздела «Практические следствия» — `examples/batch_service_optimal_bmax.py`, `examples/batch_service_dominant_pole_baseline.py`.